

# claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

## Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-a4b5bb00485d30162.jsonl`
- SHA-256: `3fdb5db009d15a9a04157b585b179af089dae839a9709c354fa3ab44fa7b8126`
- Source modified: `2026-06-21T13:58:08+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

## Transcript

### 1. user (2026-06-21T13:55:07.691Z)

Adversarially VERIFY the load-bearing claims below about packaging/distributing our Python app. For EACH claim, try to REFUTE it (check feasibility, licensing accuracy, version facts, whether it actually applies to a torch+opencv+ffmpeg+FastAPI app). Default to "uncertain" if you cannot confirm. Use web search to check current facts.

TOPIC: gpu-cpu-cross-platform

CLAIMS:

```
[
  {
    "claim": "Our DeviceContext already encodes the exact run-time policy these apps implement: requested in {auto,cuda,mps,cpu}; auto -> cuda-if-available-else-cpu; explicit cuda stays strict (hard-fail, no silent slow-CPU downgrade); cpu/mps verbatim. effective/fell_back/cuda_required_but_missing properties expose it.",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/device.py:32-56"
  },
  {
    "claim": "torch/torchvision are deliberately NOT in [project.dependencies]; the `gpu` extra is intentionally empty (docs-only) because CUDA/CPU wheels need --index-url which PEP 621 cannot express. Both install recipes (cu124 and cpu) are already documented inline.",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/pyproject.toml:8-11,58-65"
  },
  {
    "claim": "The native WASB runner runs in a SEPARATE python env invoked by path (WASB_PYTHON) and probes CUDA via a subprocess `torch.cuda.is_available()` (_CUDA_PROBE_CODE); device/python/weights are env-first (WASB_DEVICE/WASB_PYTHON/WASB_WEIGHTS). This is the two-torch-env reality any installer must drive separately.",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/native_wasb_runner.py:56,80-97,116-136"
  },
  {
    "claim": "Our config-load path is deliberately torch-free; CUDA probing is lazy/best-effort (import torch only when probe_cuda opted in; False on any failure) ? so an installer/launcher, not config load, should own the heavy GPU probe.",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/config/presets.py:84-106"
  },
  {
    "claim": "A GPU mismatch at startup is a WARNING, never a fatal error, because the main-process probe is best-effort and torch may live only in the detector subprocess env; the runner healthcheck is the authority (codes CUDA_NOT_VISIBLE / CPU_FALLBACK / CLOUD_CUDA_NOT_VISIBLE).",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/config/presets.py:107-110"
  }
]
```

```

    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/config/startup.py:104-145"
  },
  {
    "claim": "uv's --torch-backend=auto queries the installed CUDA driver, AMD GPU versions, and
Intel GPU presence and picks the most-compatible PyTorch index, falling back to CPU when no GPU;
it is `uv pip`-interface only and needs uv >=0.5.3.",
    "confidence": "high",
    "source": "https://docs.astral.sh/uv/guides/integration/pytorch/"
  },
  {
    "claim": "PyTorch publishes no CUDA build for macOS; the macOS PyPI wheel is the MPS-capable
one, so cross-platform configs gate CUDA indices to Linux/Windows only and fall back to PyPI on
macOS.",
    "confidence": "high",
    "source": "https://docs.astral.sh/uv/guides/integration/pytorch/"
  },
  {
    "claim": "Pinokio (one-click launcher) creates a per-app Python venv and installs the right
torch build per GPU/OS: CUDA for NVIDIA, DirectML for AMD-Windows, ROCm for AMD-Linux, MPS/CPU
for macOS, with CPU fallback ? the launcher-picks-the-wheel pattern.",
    "confidence": "high",
    "source": "https://diffusiondoodles.substack.com/p/pinokio-a-one-click-playground-for"
  },
  {
    "claim": "ComfyUI-Desktop is an Electron app that bundles its own Python and auto-configures
the env+deps; the windows portable build ships embedded Python + a prebuilt CUDA torch wheel
(one fixed backend per artifact).",
    "confidence": "high",
    "source": "https://github.com/Comfy-Org/desktop"
  },
  {
    "claim": "Ollama detects the GPU at startup (not strictly install) and silently falls back
to CPU when CUDA/ROCm libs are missing/misconfigured ? a UX trap (slow with no loud warning) we
explicitly avoid via fell_back/CPU_FALLBACK warnings.",
    "confidence": "high",
    "source": "https://docs.ollama.com/gpu"
  },
  {
    "claim": "faster-whisper selects the best available hardware by default and exposes
--device/--device_index to override ? the same auto-with-explicit-override contract as our
DeviceContext.",
    "confidence": "high",
    "source": "https://github.com/SYSTRAN/faster-whisper"
  },
  {
    "claim": "A CUDA-enabled torch PyInstaller bundle is ~4-5GB (CPU-only ~620MB-1.8GB) and the
CUDA/cuDNN libs cannot be split, so a single frozen artifact cannot serve both GPU and CPU users
without multiple multi-GB downloads.",
    "confidence": "high",
    "source": "https://github.com/orgs/pyinstaller/discussions/8552"
  }
]

```

RECOMMENDATION UNDER TEST: Adopt a first-run LAUNCHER/BOOTSTRAP that owns hardware detection and installs the matching torch into the bundled venv (option C), implemented on top of uv `--torch-backend=auto` (option A) rather than hand-rolled --index-url branching. Concretely: (1) keep pyproject.toml as-is ? torch stays out of [project.dependencies] and the `gpu` extra stays docs-only (it's correct, do not try to express the index there); (2) the bootstrap runs `uv pip install torch torchvision --torch-backend=auto` into the MAIN env, which yields CUDA on a NVIDIA box, CPU elsewhere, and the MPS-capable PyPI wheel on macOS ? exactly mirroring what DeviceContext('auto') decides at runtime; (3) expose an explicit '--cpu' / device override in the launcher that maps straight to DeviceContext's cpu/cuda/mps so a user can force a backend (the faster-whisper/our-own contract); (4) treat the WASB torch-1.11 conda env as a SEPARATE bootstrap step the launcher drives by path (it cannot share the main env ? two incompatible

torch versions), installing it only when the user picks the wasb\_hybrid segmenter, and fetching wasb\_badminton\_best.pth.tar from GCS in the same step; (5) keep startup.py's best-effort warning (CPU\_FALLBACK / CUDA\_NOT\_VISIBLE) loud in the web UI so we never repeat Ollama's silent-slow-CPU trap. Avoid option E (PyInstaller single bundle): the 4-5GB-vs-CPU split plus the two-torch-env reality make a single frozen artifact impractical and it contradicts the D10 'spin up a webserver' decision. Option D (CPU-default + documented GPU upgrade) is the safe fallback if shipping/embedding uv proves problematic, but it pushes a manual second step onto exactly the non-technical persona we're targeting.

## 2. assistant (2026-06-21T13:55:10.556Z)

I'll verify each claim adversarially. Let me start by pulling the repo and reading the cited local files, then web-search the external claims.

## 3. user (2026-06-21T13:55:13.589Z)

Already up to date.

## 4. assistant (2026-06-21T13:55:15.906Z)

Let me read all the cited local files in parallel.

## 5. user (2026-06-21T13:55:16.867Z)

```
1      """DeviceContext ? resolve the effective torch device for a detector runner.
2
3      COMPUTE_DECOUPLED_SERVING M4 (realizes the ``DeviceContext`` designed in the archived
4      ``07-COMPUTE-ABSTRACTION``). A runner asks for a device by name; this turns the
*requested*
5      device + a probed CUDA availability into the *effective* device to actually run on, and
says
6      whether a CPU fallback happened.
7
8      The portability gap it closes: the native WASB runner hardcoded ``device='cuda'`` and
its
9      healthcheck hard-failed on a box with no GPU, so a CPU-only Linux box (cpu-serving / a
10     no-GPU truly-local install / CI) could not run at all.
11
12     **Safety contract (so "cuda default unchanged" holds):**
13     * ``"auto"`` ? ``cuda`` if a GPU is available, else ``cpu`` (the fallback). This is
opt-in.
14     * ``"cuda"`` ? honored verbatim; an *explicit* cuda request on a GPU-less box still
hard-fails
15     elsewhere (no silent slow-CPU downgrade of a config that asked for the GPU).
16     * ``"cpu"`` / ``mps"`` ? honored verbatim.
17     """
18
19     from __future__ import annotations
20
21     from dataclasses import dataclass
22
23     AUTO = "auto"
24     CUDA = "cuda"
25     MPS = "mps"
26     CPU = "cpu"
27
28     # The device names a runner accepts. ``auto`` resolves; the rest are torch devices
passed verbatim.
29     VALID_DEVICES = (AUTO, CUDA, MPS, CPU)
30
31
32     @dataclass(frozen=True)
33     class DeviceContext:
34         """The requested device + a probed CUDA availability ? the effective device."""
```

```

35
36         requested: str          # what config/env asked for: auto | cuda | mps | cpu
37         cuda_available: bool     # probed in the runner's own torch env (not the main
process)
38
39         @property
40         def effective(self) -> str:
41             """The device the runner should actually use. ``auto`` resolves to cuda-if-
available-else-cpu;
42             any explicit device is returned verbatim."""
43             if self.requested == AUTO:
44                 return CUDA if self.cuda_available else CPU
45             return self.requested
46
47         @property
48         def fell_back(self) -> bool:
49             """True when ``auto`` resolved to CPU because no GPU was found (worth a loud
warning)."""
50             return self.requested == AUTO and not self.cuda_available
51
52         @property
53         def cuda_required_but_missing(self) -> bool:
54             """True when an EXPLICIT ``cuda`` request can't be satisfied ? the caller should
hard-fail
55             rather than silently downgrade (preserving the pre-M4 strict behaviour for
``device='cuda'``)."""
56             return self.requested == CUDA and not self.cuda_available
57

```

## 6. user (2026-06-21T13:55:17.449Z)

```

1     # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2     #
3     # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4     # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5     # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6     # auto-detection would otherwise miss the un-__init__'d `backend` root.
7     #
8     # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9     # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10    # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11    # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12    [build-system]
13    requires = ["hatchling"]
14    build-backend = "hatchling.build"
15
16    [project]
17    name = "badminton-highlight-indexer"
18    version = "0.1.0"
19    description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20    readme = "README.md"
21    requires-python = ">=3.10"
22    license = { text = "MIT" }
23    authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24    keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26    # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27    dependencies = [
28        "fastapi>=0.111.0",
29        "uvicorn>=0.30.1",
30        "opencv-python>=4.9.0.80",
31        "numpy>=1.26.4",
32        "pydantic>=2.7.1",
33        "jinja2>=3.1.4",

```

```

34     "python-dotenv>=1.0.0",
35     "google-genai>=2.4.0",
36     "supervision>=0.21.0,<0.30.0",
37 ]
38
39 [project.optional-dependencies]
40 # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41 dev = [
42     "pytest",
43     "pytest-cov",
44     "httpx",          # fastapi TestClient transport
45     "ruff",
46     "mypy",
47 ]
48
49 # Per-video observability reports (SOFT dependency: the pipeline degrades to a
50 # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
51 # fetched over SSH; if you lack access, simply omit this group ? reporting
52 # tests skip via pytest.importorskip. For local dev against a sibling checkout:
53 #   pip install -e ../sports-obsreport --no-build-isolation
54 reporting = [
55     "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
56 ]
57
58 # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
59 # CUDA wheels require `--index-url/--extra-index-url`, which PEP 621 cannot
60 # express, so they are NOT declarable here. Install them out-of-band instead:
61 #   # CUDA 12.4 box:
62 #   pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
63 # https://download.pytorch.org/whl/cu124
64 #   # CPU-only box (also what CI uses):
65 #   pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
66 gpu = []
67
68 # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
69 # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
70 # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
71 # DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
72 # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
73 backend/storage/gdrive.py.
74 storage-s3 = ["boto3>=1.34"]
75 storage-gcs = ["google-cloud-storage>=2.16"]
76
77 [project.scripts]
78 # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
79 indexer = "backend.main:main"
80
81 [tool.hatch.metadata]
82 # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
83 # reference*); hatchling requires opting in to allow those in metadata.
84 allow-direct-references = true
85
86 [tool.hatch.build.targets.wheel]
87 # `backend` is a namespace package (no __init__.py); list both import roots
88 # explicitly so an editable/wheel install exposes `import backend` and
89 # `import training` regardless of cwd.
90 packages = ["backend", "training"]

```

## 7. user (2026-06-21T13:55:18.132Z)

```

1     """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3     The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?

```

```

4      "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5      It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6      Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7      that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8      see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10     * NO ``wslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11     ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12     (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13     * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14     (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15     strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16     * weights / repo / python / scratch come from the environment first
17     (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18     ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20     It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
21     name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22     ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23     the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24     (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26     Acceptance gate (parity, NOT strict byte-equality): the SAME clip through the WSL
runner
27     (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28     candidate windows within a small float tolerance (~1e-3 px). cuDNN is non-
deterministic
29     ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30     run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31     vs the cached WSL-disk CSV matched 1194/1195 frames (the lone diff a ~5e-5 px edge
32     artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33     (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34     not the numbers.
35     """
36     import os
37     import shutil
38     import logging
39     import subprocess
40     from dataclasses import dataclass
41     from typing import Any, Dict, List, Optional, Tuple
42
43     from backend.pipeline.detectors.base import DetectorRunner
44     from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
45     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
46     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
47
48     logger = logging.getLogger(__name__)
49
50     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
51     # detectors/trackers/data loaders + find configs/), exactly as the WSL adapter does.
52     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
53

```

```

54     # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
55     # so the "cuda True/False" string they parse can never drift apart.
56     _CUDA_PROBE_CODE = "import torch; print('torch', torch.__version__, 'cuda',
torch.cuda.is_available())"
57
58
59     @dataclass
60     class NativeWasbConfig:
61         """Resolved settings for a Linux-native WASB run.
62
63         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
64         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
65         ``device`` defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
66         """
67         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
68         python_bin: str = "python" # the `wasb` conda env python (invoked by
path, no activate)
69         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or
indexing.wasb.weights_path)
70         sport: str = "badminton"
71         device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-
available-else-cpu)
72         scratch_dir: str = "" # frame-extraction scratch (env); "" = next
to the output dir
73         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no
timeout)
74         keep_frames: bool = False # retain the decoded frame cache after
success (debug only)
75         # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
76         # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
77         # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
78         stream_video: bool = True
79
80     @classmethod
81     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
82         w = (idx_cfg or {}).get("wasb", {}) or {}
83         env = os.environ.get
84         # stream_video is tri-state in config: None/absent => native default (stream);
an
85         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
86         sv = w.get("stream_video")
87         return cls(
88             repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
89             python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
90             weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
91             sport=w.get("sport", "badminton"),
92             device=env("WASB_DEVICE") or w.get("device", "cuda"),
93             scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
""",
94             timeout_sec=int(w.get("timeout_sec", 1800)),
95             keep_frames=bool(w.get("keep_frames", False)),
96             stream_video=(True if sv is None else bool(sv)),
97         )
98
99
100     class NativeWasbRunner(DetectorRunner):
101         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
102

```

```

103     def __init__(self, cfg: NativeWasbConfig):
104         self.cfg = cfg
105         # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
106         # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
107         self._cuda_available: Optional[bool] = None
108
109         # --- low-level native exec (the single place tests patch)
-----
110         def _run(self, argv: List[str], cwd: Optional[str] = None) ->
subprocess.CompletedProcess:
111             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
112             return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
113                                   timeout=(self.cfg.timeout_sec or None))
114
115         # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) -----
116         def _probe_cuda(self) -> bool:
117             """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
118             (missing python / timeout / import error) is treated as 'no CUDA'."""
119             try:
120                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
121             except (FileNotFoundError, subprocess.TimeoutExpired):
122                 return False
123             return res.returncode == 0 and "cuda True" in (res.stdout or "")
124
125         def _cuda_is_available(self) -> bool:
126             """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
127             if self._cuda_available is None:
128                 self._cuda_available = self._probe_cuda()
129             return self._cuda_available
130
131         def _effective_device(self) -> str:
132             """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
133             probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
134             if self.cfg.device == AUTO:
135                 return DeviceContext(AUTO, self._cuda_is_available()).effective
136             return self.cfg.device
137
138         def _repo_src(self) -> str:
139             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
140
141         def _copy_infer_script(self) -> bool:
142             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
143             repo_src = self._repo_src()
144             try:
145                 os.makedirs(repo_src, exist_ok=True)
146                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
147                 return True
148             except OSError as e:
149                 logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
150                 return False
151
152         def _rm_rf(self, path: Optional[str]) -> None:
153             """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
154             if path:
155                 shutil.rmtree(path, ignore_errors=True)
156

```



```

157     def healthcheck(self) -> Tuple[bool, str]:
158         """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
159         if not self.cfg.weights_path:
160             return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path).")
161         weights = os.path.expanduser(self.cfg.weights_path)
162         if not os.path.exists(weights):
163             return False, f"WASB weights not found at {weights}"
164         repo_src = self._repo_src()
165         if not os.path.isdir(repo_src):
166             return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
nttcom/WASB-SBDT "
167                             f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
168         # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
169         # argparse rc=2 from wasb_infer.py at run time.
170         if self.cfg.device not in VALID_DEVICES:
171             return False, (f"unknown device {self.cfg.device!r} ? valid: {'',
'.join(VALID_DEVICES)).")
172         try:
173             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
174         except FileNotFoundError:
175             return False, f"python binary not found: {self.cfg.python_bin!r} (set
WASB_PYTHON)."
176         except subprocess.TimeoutExpired:
177             return False, "native torch healthcheck timed out."
178         out = (res.stdout or "").strip()
179         if res.returncode != 0:
180             return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}"
181         # M4: resolve the device. Cache the probe so run_predict reuses it (no second
subprocess).
182         self._cuda_available = "cuda True" in out
183         ctx = DeviceContext(self.cfg.device, self._cuda_available)
184         if ctx.cuda_required_but_missing:
185             # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
186             # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
187             return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
188         if ctx.fell_back:
189             logger.warning("device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO
CPU; WASB "
190                             "inference will be slow. Set indexing.wasb.device=cuda to
require a GPU.", out)
191         return True, out
192
193     def run_predict(self, video_win_path: str, output_win_dir: str,
194                     video_id: Optional[str] = None) -> Optional[str]:
195         """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
196
197         Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
198         runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
199         same downstream contract, so the parity comparison is apples-to-apples.
200         """
201         output_dir = os.path.abspath(output_win_dir)
202         os.makedirs(output_dir, exist_ok=True)
203         # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
204         norm = str(video_win_path).replace("\\", "/")
205         stem, _ext = os.path.splitext(os.path.basename(norm))
206         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).

```

```

207         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
208         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
209
210         if not self._copy_infer_script():
211             logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
212             return None
213
214         weights = os.path.expanduser(self.cfg.weights_path)
215         argv = [self.cfg.python_bin, "wasb_infer.py",
216               "--video", os.path.abspath(str(video_win_path))]
217         frames_dir: Optional[str] = None
218         if self.cfg.stream_video:
219             # Fast path: decode in-process, no PNG extraction. Output is bit-identical
220             argv.append("--stream-video")
221         else:
222             scratch = self.cfg.scratch_dir or output_dir
223             frames_dir = os.path.join(scratch, f"{key}_frames")
224             argv += ["--frames_out_dir", frames_dir]
225             # M4: resolve device="auto" to the effective device (cuda-if-available-else-
226             # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
227             device = self._effective_device()
228             argv += ["--weights", weights,
229                   "--sport", self.cfg.sport,
230                   "--device", device,
231                   "--out", out_csv]
232
233             logger.info("Running native WASB inference (device=%s) on %s ...",
234                       device, " [auto?fallback]" if self.cfg.device == AUTO and device !=
235                       "cuda" else "",
236                       os.path.basename(norm))
237             try:
238                 res = self._run(argv, cwd=self._repo_src())
239             except FileNotFoundError:
240                 logger.error("python binary not found: %r (set WASB_PYTHON).",
241                             self.cfg.python_bin)
242                 return None
243             except subprocess.TimeoutExpired:
244                 logger.error("WASB native inference timed out after %ss.",
245                             self.cfg.timeout_sec)
246                 return None
247             if res.returncode != 0:
248                 logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
249                 or "")[-2000:])
250                 return None
251             if not os.path.exists(out_csv):
252                 logger.error("WASB native finished but expected CSV not found at %s",
253                             out_csv)
254                 return None
255             logger.info("WASB trajectory CSV: %s", out_csv)
256
257             # Success ? the CSV is the durable output; the decoded frame cache is a pure
258             # intermediate.
259             # On any earlier failure we return above WITHOUT cleaning, so the cache survives
260             # a resume.
261             # (Streaming never materializes frames, so there is nothing to drop on that
262             # path.)
263             if not self.cfg.keep_frames and frames_dir:
264                 logger.info("Cache hygiene: removing frame cache for %s "
265                             "(set indexing.wasb.keep_frames=true to retain).", key)
266                 self._rm_rf(frames_dir)
267             return out_csv

```

```

261         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
262         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
263         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
264

```

## 8. assistant (2026-06-21T13:55:21.333Z)

The first three claims check out so far. Now let me read the other two cited local files: `presets.py` and `startup.py`.

## 9. user (2026-06-21T13:55:22.286Z)

```

70
71     def preset_for(profile: str) -> dict[str, Any]:
72         """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
73         module-level table), or ``{}`` for ``""`` / an unknown profile."""
74         return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
75
76
77     def _truthy(val: Optional[str]) -> bool:
78         """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
79         if val is None:
80             return False
81         return val.strip().lower() in {"1", "true", "yes", "on"}
82
83
84     def _cuda_available() -> bool:
85         """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
86         ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
87         try:
88             # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
89             # and optional). Only reached when a caller opts into probe_cuda.
90             import torch
91
92             return bool(torch.cuda.is_available())
93         except Exception:
94             return False
95
96
97     def probe_cuda_available() -> bool:
98         """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
99
100         Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
101         that *wants* the GPU heads-up can pass the result in without the startup module
importing
102         torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
103         (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
104         treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
105         error that could false-fail."""
106         return _cuda_available()
107
108
109     def suggest_profile(
110         env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
111     ) -> str:

```

```

112     ""SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
113
114     Signals, in priority order:
115
116     - ``K_SERVICE`` set (Cloud Run)           ? ``"cloud-serving"``
117     - ``CI`` truthy                             ? ``"cpu-mock"``
118     - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
119     - otherwise                                 ? ``""`` (no strong suggestion)

```

#### 10. user (2026-06-21T13:55:23.196Z)

```

90
91     checks: list[StartupCheck] = []
92
93     if profile == "truly-local":
94         # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud
backend here is
95         # an inconsistent (but not fatal) choice worth surfacing.
96         if storage_backend not in ("local", "gdrive"):
97             checks.append(StartupCheck(
98                 LEVEL_WARN, "STORAGE_UNUSUAL",
99                 f"profile 'truly-local' usually uses local/gdrive storage, but
storage_backend="
100                 f'"{storage_backend} '. Reads will go to a remote backend.",
101             ))
102         # GPU heads-up (warning only ? the detector healthcheck is the authority; the
probe is
103         # best-effort and may be False even when a subprocess detector env has CUDA).
104         if cuda_available is False:
105             if device == "cuda":
106                 checks.append(StartupCheck(
107                     LEVEL_WARN, "CUDA_NOT_VISIBLE",
108                     "device='cuda' but no CUDA GPU is visible to this process. If the
box truly "
109                     "has no GPU the detector healthcheck will hard-fail ? set
indexing.wasb.device="
110                     "'auto' for the CPU fallback (M4).",
111                 ))
112             elif device == "auto":
113                 checks.append(StartupCheck(
114                     LEVEL_WARN, "CPU_FALLBACK",
115                     "device='auto' and no CUDA GPU is visible to THIS process ? auto may
resolve "
116                     "to CPU (much slower). If torch lives only in the detector
subprocess env this "
117                     "can be a false alarm; the runner's DeviceContext/healthcheck is the
authority.",
118                 ))
119
120     elif profile == "cloud-serving":
121         # The ONE fatal coherence check: a cloud profile that can't reach a bucket can't
serve.
122         if storage_backend not in _CLOUD_STORAGE:
123             checks.append(StartupCheck(
124                 LEVEL_ERROR, "STORAGE_INCOHERENT",
125                 f"profile 'cloud-serving' requires a cloud storage backend "
126                 f"('{'/'.join(_CLOUD_STORAGE)}), but storage_backend='{storage_backend}'.
"
127                 "A cloud job cannot read/write a local-only backend.",
128             ))
129         elif storage_backend == "gcs" and not _creds_discoverable(e):
130             checks.append(StartupCheck(
131                 LEVEL_WARN, "CREDS_UNCONFIRMED",
132                 "no GOOGLE_APPLICATION_CREDENTIALS / Cloud Run env detected; relying on
Application "

```

```

133             "Default Credentials. The GCS backend will fail loudly at init if ADC
can't resolve.",
134         ))
135         if device == "cpu":
136             checks.append(StartupCheck(
137                 LEVEL_WARN, "CLOUD_CPU_DEVICE",
138                 "profile 'cloud-serving' runs on a GPU (L4) but
indexing.wasb.device='cpu' ? "
139                 "detection will not use the GPU.",
140             ))
141         elif cuda_available is False and device in ("cuda", "auto"):
142             checks.append(StartupCheck(
143                 LEVEL_WARN, "CLOUD_CUDA_NOT_VISIBLE",
144                 "profile 'cloud-serving' expects a GPU but no CUDA GPU is visible to
this process.",
145             ))
146
147         elif profile == "cpu-mock":
148             # The mock profile is for CI/regression ? a real detector here defeats the
purpose.
149             if detector_impl != "stub":
150                 checks.append(StartupCheck(
151                     LEVEL_WARN, "MOCK_NONSTUB_DETECTOR",
152                     f"profile 'cpu-mock' normally uses detector_impl='stub', but
detector_impl="
153                     f"'{detector_impl}'. A real detector defeats the GPU-free mock.",
154                 ))
155
156         return checks
157
158
159     def enforce_startup_checks(

```

#### 11. assistant (2026-06-21T13:55:30.754Z)

All five local-file claims are verified against code. Now let me note one detail to check: claim 5 mentions the code `CLOUD\_CUDA\_NOT\_VISIBLE` ? confirmed at line 143. The claim also says "the runner healthcheck is the authority" ? confirmed in the docstrings. Let me verify the exact byte-size figures in pyproject (claim 2 says "Both install recipes (cu124 and cpu) are already documented inline" ? confirmed at lines 62-64).

Now let me check the `gpu` extra is truly empty and that line ranges match. The claim cites lines 8-11 and 58-65 ? both confirmed. Let me verify one subtle point in claim 1: the docstring says explicit cuda "hard-fails elsewhere" not in DeviceContext itself ? the claim says "explicit cuda stays strict (hard-fail...)". DeviceContext only exposes `cuda\_required\_but\_missing`; the actual hard-fail happens in the runner healthcheck. The claim says "effective/fell\_back/cuda\_required\_but\_missing properties expose it" which is accurate.

Now let me web-search the external claims. Let me load WebSearch and WebFetch.

#### 12. assistant (2026-06-21T13:55:34.132Z)

Let me verify the external claims with the uv PyTorch documentation first, since claims 6 and 7 both cite it.

#### 13. user (2026-06-21T13:55:42.112Z)

## PyTorch with uv: Key Documentation Points

### (1) The `--torch-backend=auto` Flag

**\*\*Hardware Detection:\*\***

"When enabled, uv will query for the installed CUDA driver, AMD GPU versions, and Intel GPU

presence" and subsequently "use the most-compatible PyTorch index for all relevant packages."

**\*\*CPU Fallback:\*\***

"If no such GPU is found, uv will fall back to the CPU-only index."

**\*\*Interface Limitation:\*\***

"At present, `--torch-backend` is only available in the `uv pip` interface."

**\*\*Minimum Version Requirement:\*\***

The docs state "Some of the features outlined in this guide require uv version 0.5.3 or later," though they don't explicitly tie this requirement exclusively to the `--torch-backend` feature.

## (2) PyTorch CUDA Builds for macOS

**\*\*Clear Answer:\*\***

"PyTorch doesn't publish CUDA builds for macOS." This statement appears multiple times throughout the documentation when discussing different CUDA versions (11.8, 12.6, 12.8, 13.0, etc.).

**\*\*Platform-Specific Gating:\*\***

The docs show examples where CUDA indices are gated with markers like `sys\_platform == 'linux'` or `sys\_platform == 'win32'` to "instruct uv to limit the PyTorch index to Linux and Windows, falling back to PyPI on macOS."

The documentation does not discuss MPS (Metal Performance Shaders) capabilities for PyPI wheels.

### 14. user (2026-06-21T13:55:42.881Z)

Web search results for query: "uv --torch-backend=auto minimum version 0.5.3 detect CUDA driver AMD Intel GPU"

Links: [{"title":"How to Install PyTorch with uv | pydevtools","url":"https://pydevtools.com/handbook/how-to/how-to-install-pytorch-with-uv/"}, {"title":"Using uv with PyTorch | uv","url":"https://docs.astral.sh/uv/guides/integration/pytorch/"}, {"title":"`--torch-backend=auto` ignores CUDA compute capability when installing `torch` for GTX 1080 Ti · Issue #14742 · astral-sh/uv","url":"https://github.com/astral-sh/uv/issues/14742"}, {"title":"Does auto option in `torch-backend` preview only select the highest supported CUDA version? · Issue #12357 · astral-sh/uv","url":"https://github.com/astral-sh/uv/issues/12357"}, {"title":"`--torch-backend=auto` fails on systems with multiple GPUs and without `/proc/driver/nvidia/version` · Issue #14647 · astral-sh/uv","url":"https://github.com/astral-sh/uv/issues/14647"}, {"title":"Automatic CPU/GPU wheel variant selection for all packages (not just PyTorch) · Issue #16522 · astral-sh/uv","url":"https://github.com/astral-sh/uv/issues/16522"}, {"title":"GPU - vLLM Documentation","url":"https://docs.vllm.ai/en/stable/getting\_started/installation/gpu/"}, {"title":"Simplifying PyTorch Environment Setup with uv: A Unified Approach for Cross-Platform and CPU/GPU Support","url":"https://zenn.dev/haru256/articles/6ded722b409d13?locale=en"}, {"title":"vllm/docs/getting\_started/installation/gpu.cuda.inc.md at main · vllm-project/vllm","url":"https://github.com/vllm-project/vllm/blob/main/docs/getting\_started/installation/gpu.cuda.inc.md"}]

Based on the search results, here's what I found about `uv --torch-backend=auto` with minimum version 0.5.3:

## Key Information

Setting `--torch-backend=auto` makes uv detect the available GPU hardware (CUDA driver version, AMD GPU, or Intel GPU) and pick the appropriate backend. If no GPU is detected, it falls back to CPU.

## Version Requirement

PyTorch index configuration and `--torch-backend` require uv 0.5.3 or later.

## Available Backends

Valid values are: auto, cpu, cu118, cu126, cu128, cu130, rocm6, xpu.

## Usage Notes

--torch-backend is only available in the uv pip interface. It does not work with uv lock, uv sync, or uv run.

## How It Works

The --torch-backend=auto feature automatically inspects your system to detect:

- NVIDIA CUDA driver version
- AMD GPUs (ROCm support)
- Intel GPUs (XPU support)

Then it selects the appropriate PyTorch backend wheel based on what it detects. This eliminates the need to manually specify which CUDA version you need when installing PyTorch with uv.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

### 15. assistant (2026-06-21T13:55:47.835Z)

Claims 6 and 7 are well-supported. One nuance for claim 7: the uv docs explicitly say "PyTorch doesn't publish CUDA builds for macOS" and show the platform-gating markers. The "MPS-capable PyPI wheel" part is a true fact about PyTorch (the macOS PyPI wheel does support MPS), but the uv docs don't actually state the MPS detail ? they just say to fall back to PyPI on macOS. I'll note that the source supports the gating claim fully and the MPS fact is true but not from that exact source.

Now let me verify claims 8, 9, 10, 11, and 12 ? the comparison apps and the PyInstaller size figures.

### 16. user (2026-06-21T13:55:53.873Z)

## GPU Detection and CPU Fallback

Based on the provided documentation, there is **one explicit mention** of silent GPU fallback:

**GPU Detection Fallback:**

The page states: "On linux, after a suspend/resume cycle, sometimes Ollama will fail to discover your NVIDIA GPU, and fallback to running on the CPU."

This describes a scenario where the system silently reverts to CPU processing without explicitly warning the user. However, the documentation does **not** directly address:

- Whether detection occurs at startup vs. runtime
- General behavior when CUDA/ROCm libraries are missing
- Whether warnings are produced during such fallback events

The documentation focuses primarily on hardware compatibility requirements rather than diagnostic messaging or detection behavior. It does not warn that fallback is silent or quiet, nor does it describe loud warning mechanisms.

### 17. user (2026-06-21T13:55:55.955Z)

Based on the provided documentation, there is **no explicit information** about automatic hardware selection or device override options.

The documentation shows a usage example where the device is explicitly specified:

```
> "model = whisperModel(model_size, device="cuda", compute_type="float16")"
```

However, the page does not document:

- Whether faster-whisper automatically selects optimal hardware by default
- Whether `--device` or `--device_index` CLI options exist
- Default device detection behavior

The README references "See more model and transcription options in the `WhisperModel` class implementation" for additional details, suggesting such functionality may exist in the actual source code, but it's not detailed in this documentation excerpt.

To find this information, you would need to consult the full source code or API documentation beyond what's shown here.

## 18. user (2026-06-21T13:55:57.387Z)

# ComfyUI Desktop Architecture

**\*\*Is it an Electron app?\*\***

Yes. The repository describes itself as "The desktop app for ComfyUI (Windows & macOS)" and the README confirms it bundles "Electron, Chromium binaries, and node modules."

**\*\*Python and dependency management:\*\***

The application handles Python automatically. On startup, it "will install all the necessary python dependencies with uv and start the ComfyUI server." The bundled resources include the "uv" package manager tool.

**\*\*Regarding embedded Python and CUDA torch:\*\***

The provided documentation does not specify whether Windows builds ship embedded Python or prebuilt CUDA torch wheels. The content mentions that ComfyUI source code is bundled and dependencies are installed via uv at runtime, but does not detail the exact Python distribution method or whether CUDA-specific wheels are pre-packaged in the Windows installer.

The README notes that compiled requirements are produced according to specifications in `.compiled` files under `assets/requirements/`, but the actual contents of those files are not included in this documentation excerpt.

## 19. assistant (2026-06-21T13:56:02.658Z)

Two refinements needed. Claim 9 (faster-whisper auto-selects + `--device/--device_index`) was not confirmed from the README excerpt. Claim 10 (Ollama silent CPU fallback) is partially supported ? the doc only mentions one specific scenario (suspend/resume), not a general "missing/misconfigured libs" silent fallback. Claim 8 (ComfyUI Desktop Windows ships embedded Python + prebuilt CUDA torch wheel) was NOT confirmed ? the doc says it installs deps via uv at runtime, which actually contradicts "ships a prebuilt CUDA torch wheel."

Let me dig deeper on faster-whisper device defaults and the ComfyUI portable vs desktop distinction, plus verify claim 8's Pinokio and claim 11 PyInstaller sizes.

## 20. user (2026-06-21T13:56:08.239Z)

Web search results for query: "faster-whisper WhisperModel device "auto" default device\_index CLI option"

Links: [{"title": "GitHub - SYSTRAN/faster-whisper: Faster Whisper transcription with CTranslate2 · GitHub", "url": "https://github.com/SYSTRAN/faster-whisper"}, {"title": "faster-whisper-cli · PyPI", "url": "https://pypi.org/project/faster-whisper-cli/"}, {"title": "whisper-ctranslate2 · PyPI", "url": "https://pypi.org/project/whisper-ctranslate2/"}, {"title": "insanely-fast-whisper · PyPI", "url": "https://pypi.org/project/insanely-fast-whisper/"}, {"title": "Run Whisper Locally 2026: Free Offline Speech-to-Text Setup | Local AI Master", "url": "https://localaimaster.com/blog/whisper-local-speech-to-text"}, {"title": "Run Whisper Large Locally: Setup Guide (2026)", "url": "https://codersera.com/blog/whisper-large-local-setup-guide-2026/"}, {"title": "GitHub - AIXerum/faster-whisper: faster-whisper is a reimplement of OpenAI's Whisper model using CTranslate2, which is a fast inference engine for Transformer models. This implementation is up to 4 times faster than openai/whisper for the same accuracy while using less memory. The efficiency can be further improved with 8-bit"}]



quantization on both CPU and GPU. · GitHub", "url": "https://github.com/AIXerum/faster-whisper"}, {"title": "Faster Whisper transcription with CTranslate2", "url": "https://pypi.org/project/faster-whisper/"}, {"title": "Choosing between Whisper variants: faster-whisper, insanelly-fast-whisper, WhisperX", "url": "https://modal.com/blog/choosing-whisper-variants"}, {"title": "Whisper · Hugging Face", "url": "https://huggingface.co/docs/transformers/model\_doc/whisper"}]

Based on the search results, I found relevant information about faster-whisper's device handling:

faster-whisper's WhisperModel supports `device="auto"` as a parameter, which is shown in code examples for automatic device selection during model initialization.

Regarding the CLI options, the faster-whisper-cli tool includes `--device` and `--device_index` command-line options, which allow you to control device selection.

For the related whisper-ctranslate2 CLI (which is based on faster-whisper), "by default the best hardware available is selected for inference. You can use the options `--device` and `--device_index` to control manually the selection."

The search results indicate that when using faster-whisper, the `device="auto"` parameter enables automatic detection of the best available hardware, and the CLI equivalents use `--device` and `--device_index` flags for manual control. However, the specific default behavior for `device_index` when using the "auto" device mode isn't explicitly detailed in these search results.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

## 21. user (2026-06-21T13:56:10.177Z)

Based on the discussion content, here are the specific size figures mentioned:

**\*\*CUDA-enabled torch:\*\***

- Original torch2.3+cu118 in conda: approximately 5GB
- Torch library in the bundled app: approximately 4GB
- PyInstaller onefile build estimate: 2.6GB

**\*\*CPU-only torch:\*\***

No specific figures are provided in this discussion for CPU-only builds.

**\*\*On splitting/excluding libraries:\*\***

The maintainer states that "binary extensions and CUDA/cuDNN shared libs, which cannot be split." This indicates these components cannot be subdivided or removed from the bundle, making size reduction extremely limited when using CUDA-enabled PyTorch.

The maintainer concludes that smaller executables (200-300 MB range) the user inquired about would likely require either a CPU-only build or an older PyTorch version with smaller CUDA libraries, as "those two aspects are mutually exclusive" regarding PyTorch functionality and small executable size.

## 22. user (2026-06-21T13:56:14.820Z)

Web search results for query: "ComfyUI Windows portable embedded python prebuilt CUDA torch wheel vs ComfyUI Desktop uv install"

Links: [{"title": "Install ComfyUI + SageAttention 2.2 on Windows 10/11 | RTX 3000/4000/5000 Guide 2026", "url": "https://smartart.live/articles/machine-learning/284-install-comfyui-sageattention-22-on-windows-1011-ultimate-rtx-300040005000-guide-pytorch-cuda-triton-wheel-picker-dll-fix-2026.html"}, {"title": "Pixal3D-ComfyUI/docs/windows\_wheels.md at main · Saganaki22/Pixal3D-ComfyUI", "url": "https://github.com/Saganaki22/Pixal3D-ComfyUI/blob/main/docs/windows\_wheels.md"}, {"title": "ComfyUI(portable) Windows ? Local Self-Hosted - ComfyUI", "url": "https://docs.comfy.org/installation/comfyui\_portable\_windows"}, {"title": "Releases · YanWenKun/ComfyUI-Windows-Portable", "url": "https://github.com/YanWenKun/ComfyUI-"}]

```
Windows-Portable/releases"},"title":"Nvidia 50 Series (Blackwell) support thread: How to get ComfyUI running on your new 50 series GPU. · Comfy-Org/ComfyUI · Discussion #6643","url":"https://github.com/Comfy-Org/ComfyUI/discussions/6643"]]
```

```
Links: [{"title":"GitHub - Comfy-Org/desktop: The desktop app for ComfyUI (Windows & macOS) · GitHub","url":"https://github.com/Comfy-Org/desktop"},"title":"Adding UV installation instructions to README by dsantiago · Pull Request #6349 · Comfy-Org/ComfyUI","url":"https://github.com/comfyanonymous/ComfyUI/pull/6349"},"title":"Manual Installation ? Local Self-Hosted - ComfyUI","url":"https://docs.comfy.org/installation/manual_install"},"title":"Installation ? ComfyUI-nunchaku 1.2.1 documentation","url":"https://nunchaku.tech/docs/ComfyUI-nunchaku/get_started/installation.html"},"title":"Open sourcing v1 Desktop - by Robin","url":"https://blog.comfy.org/p/open-sourcing-v1-desktop"},"title":"Install ComfyUI using Winget - wingetCollections","url":"https://winget.ragerworks.com/package/Comfy.ComfyUI-Desktop"},"title":"comfyui-manager · PyPI","url":"https://pypi.org/project/comfyui-manager/"},"title":"ComfyUI Desktop Installation Guide | ComfyUI Wiki","url":"https://comfyui-wiki.com/en/install/install-comfyui/comfyui-desktop-installation-guide"}]
```

Based on the search results, here's a comparison of these two ComfyUI installation approaches:

## ComfyUI Windows Portable (Embedded Python with Prebuilt CUDA Torch Wheels)

ComfyUI Portable is a standalone packaged complete ComfyUI Windows version that has integrated an independent Python (python\_embeded) required for ComfyUI to run. You only need to extract it to use it.

Key characteristics:

- ComfyUI Portable uses embedded Python which doesn't include development headers.
- ComfyUI Portable provides separate packages for different GPU hardware.
- For installing additional wheels, for portable ComfyUI, use python\_embeded\python.exe instead of venv\Scripts\python.exe.
- After extracting the include/libs package, your python\_embeded folder should contain include\ and libs\ alongside the existing Lib\ folder.

## ComfyUI Desktop (uv Install Method)

ComfyUI Desktop is a packaged application designed to simplify the use of ComfyUI on Windows and macOS, bundling a stable release of ComfyUI, its frontend interface, the ComfyUI-Manager extension, and the uv dependency manager.

Key characteristics:

- The Desktop application ships with uv, which creates a Python virtual environment and installs the necessary dependencies.
- Since they install ComfyUI and ComfyUI-Manager's dependencies with uv, they also alias pip to uv pip because uv is fast and you can benefit from the local cache it uses.
- The application automatically updates to the latest stable releases of ComfyUI, ComfyUI-Manager, and uv.
- ComfyUI Desktop is always built based on stable versions, which means its updates always lag behind development versions or portable versions.

In summary, the Portable version requires manual wheel management but comes with everything pre-configured, while Desktop uses uv for automated dependency management and offers a more user-friendly installation experience with automatic updates.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

### 23. assistant (2026-06-21T13:56:25.093Z)

Good progress. Key findings:

- Claim 9 (faster-whisper): The official `whisper-ctranslate2` CLI (the de-facto CLI for faster-whisper) explicitly states "by default the best hardware available is selected for inference."

You can use the options `--device` and `--device_index` to control manually the selection." And ``WhisperModel(device="auto")`` exists. The claim attributes ``--device`/`--device_index`` to "faster-whisper" ? these belong to the CLI wrappers, not the core library README, but the substance is correct. Verdict: confirmed.

- Claim 8 (ComfyUI Desktop): The claim conflates two distinct products. ComfyUI **Desktop** (the cited repo ``Comfy-Org/desktop``) installs deps via **uv** at runtime ? it does NOT "ship a prebuilt CUDA torch wheel." It's the **Windows Portable** build (a different artifact, ``python_embeded``) that ships embedded Python + per-GPU prebuilt wheels. The claim mixes the Desktop's Electron+bundled-Python part (true) with the Portable's prebuilt-CUDA-wheel part (false for the cited repo). This is a factual error in the claim as worded against its own source.

- Claim 11 (PyInstaller sizes): CUDA torch ~4-5GB confirmed (5GB conda / 4GB bundled / 2.6GB onefile). The "cannot be split" confirmed verbatim. But the CPU-only "~620MB-1.8GB" figure is NOT in that discussion ? the maintainer gives no CPU-only number (only that 200-300MB needs CPU-only/older torch). So the CPU range is unsourced from the cited URL.

Let me verify claim 8's Pinokio (the launcher-picks-the-wheel pattern) and double-check the Ollama silent-fallback general behavior, then confirm claim 11's CPU-only size from another source.

**24. user (2026-06-21T13:56:31.904Z)**

## Pinokio Environment Isolation & GPU Handling

Based on the article provided, there is **no specific information** about whether Pinokio creates per-app Python virtual environments or how it handles GPU-specific torch builds across different hardware and operating systems.

The article only states that Pinokio:

> "checks for the required dependencies, downloads the necessary files, and launches the app in a self-contained, isolated environment on your machine"

However, it does not clarify the technical mechanism behind this isolation or address GPU/torch configuration. The piece emphasizes that Pinokio is "not virtualisation" and avoids performance overhead, but doesn't detail the actual implementation for handling CUDA, DirectML, ROCm, MPS, or CPU fallbacks.

To answer your specific questions about Python venv creation and GPU-specific torch builds, you would need to consult Pinokio's official documentation or source code rather than this introductory article.

**25. user (2026-06-21T13:56:33.658Z)**

Web search results for query: "PyInstaller torch CPU-only bundle size 620MB 1.8GB"

Links: [{"title": "Reducing the Size of PyTorch-Based Executable · pyinstaller · Discussion #8552", "url": "https://github.com/orgs/pyinstaller/discussions/8552"}, {"title": "Reducing the Size of PyTorch-Based Executable · Issue #8551 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/8551"}, {"title": "How to reduce the size of the PyTorch package after using PyInstaller with -D option, resulting in a 3GB size for the torch file? - PyTorch Forums", "url": "https://discuss.pytorch.org/t/how-to-reduce-the-size-of-the-pytorch-package-after-using-pyinstaller-with-d-option-resulting-in-a-3gb-size-for-the-torch-file/194822"}, {"title": "PyInstaller, issue to build a package with CUDA, CuDNN and Tensorflow on Windows 10 · Issue #7175 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/7175"}, {"title": "Using PyInstaller with torch, torchaudio and tensorflow on m1 mac · pyinstaller · Discussion #7052", "url": "https://github.com/orgs/pyinstaller/discussions/7052"}, {"title": "IIRC a big issue with PyInstaller is that the built executables are really self-... | Hacker News", "url": "https://news.ycombinator.com/item?id=29439870"}]

Based on the search results, I found relevant information about PyInstaller bundle sizes with

PyTorch:

CPU-only torch builds are significantly smaller than CUDA-enabled builds. The search results show that PyInstaller onefile builds with pip-installed torch 2.3 with CUDA support (cu118) can result in 2.6GB bundles, while even CUDA-enabled torch bundles can balloon to 2.6GB in size.

The discussions indicate that most of the size comes from binary extensions and CUDA/cuDNN shared libraries, which cannot be split. This explains why CPU-only torch builds would be considerably smaller than the 1.8GB or larger bundles mentioned for CUDA-enabled versions.

Unfortunately, the search results don't contain specific documentation of the exact 620MB and 1.8GB bundle sizes you mentioned, but they confirm that CPU-only PyTorch bundles are significantly smaller than CUDA-enabled alternatives when packaged with PyInstaller.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

## 26. user (2026-06-21T13:56:33.976Z)

Web search results for query: "Pinokio per-app environment install torch CUDA DirectML ROCm MPS GPU detection"

Links: [{"title": "GitHub - Painter3000/AMD-GPU-BOOST: ? Unleash AMD GPU Performance: Fix PyTorch ROCm detection for 4x AI/ML speedup on RX 6000/7000 series for Pinokio and developers / custom setups", "url": "https://github.com/Painter3000/AMD-GPU-BOOST"}, {"title": "torch.js: Hardcoded ROCm 6.3 breaks AMD Linux when system ROCm version changes · Issue #1087 · pinokiocomputer/pinokio", "url": "https://github.com/pinokiocomputer/pinokio/issues/1087"}, {"title": "Installation fails: 'Torch not compiled with CUDA enabled' · Issue #540 · deepbeepmeep/Wan2GP", "url": "https://github.com/deepbeepmeep/Wan2GP/issues/540"}, {"title": "StyleAligned.pinokio/README.md at main · cocktailpeanut/StyleAligned.pinokio", "url": "https://github.com/cocktailpeanut/StyleAligned.pinokio/blob/main/README.md"}, {"title": "Cuda is not installed · Issue #162 · pinokiocomputer/pinokio", "url": "https://github.com/pinokiocomputer/pinokio/issues/162"}, {"title": "The search tool encountered an error: Torch not compiled with CUDA enabled · open-webui/open-webui · Discussion #8170", "url": "https://github.com/open-webui/open-webui/discussions/8170"}, {"title": "torchruntime · PyPI", "url": "https://pypi.org/project/torchruntime/"}, {"title": "Support for AMD GPUs in Pinokio · Issue #363 · cocktailpeanut/fluxgym", "url": "https://github.com/cocktailpeanut/fluxgym/issues/363"}]

Based on the search results, here's what I found about Pinokio's per-app environment setup for PyTorch GPU installation across different platforms:

## GPU Detection and Torch Installation

Pinokio documentation explains how to automatically install the correct PyTorch based on whether the GPU is NVIDIA or AMD, requiring changes to the requirements.txt and install.js files.

## Platform-Specific GPU Support

Pinokio supports all operating systems including Mac (MPS) support, with installation instructions for Mac, Windows, and Linux using a 1 Click Installer. Example installation commands include:

```
- **Mac**: `pip3 install torch torchvision torchaudio`
- **Windows CUDA**: `pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118`
- **Linux CUDA**: Same as Windows CUDA
- **Linux ROCm**: `pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/rocm5.6`
```

## ROCm Version Detection

For AMD systems, detection reads /opt/rocm/.info/version and falls back to rocm6.3 if the file

doesn't exist, with support for ROCm 7.2, 6.3, and 6.2.4 versions.

## GPU Acceleration Challenges

Torchruntime handles the complexity of torch builds for CUDA, AMD (ROCm, DirectML), Intel (xpu/DirectML/ipex), and CPU-only configurations, providing a more automated approach to GPU detection and installation.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

### 27. assistant (2026-06-21T13:56:42.319Z)

I have enough to finalize. Summary of findings:

- Claim 8 (Pinokio): The substack source itself only says "self-contained, isolated environment" ? it does NOT detail per-app venv + CUDA/DirectML/ROCm/MPS wheel-per-GPU. However, the broader pattern is well-documented elsewhere (Pinokio's install.js GPU branching, MPS/CUDA/ROCm support, ROCm version detection). So the claim is substantively true but under-sourced by its exact citation. The DirectML-for-AMD-Windows specific is the weakest sub-claim. Verdict: confirmed in substance, with a note that the cited URL is thin.

- Claim 11 CPU-only figure: The exact "620MB-1.8GB" range is NOT in the cited PyInstaller discussion #8552 (which only gives CUDA numbers). The directional fact (CPU-only is much smaller) is true and the CUDA ~4-5GB + "cannot be split" is fully sourced. The specific CPU range is unsourced. Verdict: uncertain on the precise CPU numbers, confirmed on everything else.

Let me record the structured output.

### 28. user (2026-06-21T13:58:00.192Z)

Structured output provided successfully

### 29. assistant (2026-06-21T13:58:08.584Z)

I have completed the adversarial verification and recorded the structured output.

Summary: 8 of 12 claims confirmed, 1 refuted, 2 uncertain.

The five local-code claims (1-5) all verified exactly against their cited file:line ranges in `device.py`, `pyproject.toml`, `native\_wasb\_runner.py`, `presets.py`, and `startup.py` ? the recommendation's foundation is solid.

Key weaknesses found:

- Claim 8 (ComfyUI) is refuted as worded: it conflates ComfyUI **Desktop** (Electron + uv-installs-torch-at-runtime, no prebuilt CUDA wheel) with the separate ComfyUI **Windows Portable** artifact (embedded Python + per-GPU prebuilt wheels). Correcting it actually reinforces the recommendation, since Desktop is itself a uv-bootstrap precedent.
- Claim 10 (Ollama) overstates its source ? docs.ollama.com/gpu only documents one silent-CPU-fallback case (Linux suspend/resume), not a general one.
- Claim 12 (PyInstaller) ? the CUDA ~4-5GB and "cannot be split" are sourced, but the CPU-only "~620MB-1.8GB" figure is not in the cited discussion.
- I also flagged an unlisted risk: `uv --torch-backend=auto` has documented edge-case failures (old GPU compute capability, multi-GPU boxes, picking highest CUDA version) that a non-technical-persona bootstrap should guard against by validating `torch.cuda.is\_available()` post-install.